

Full Stack Developer Interview Assignment

Support Ticketing Module

Suggested timebox	Core stack	Focus
4-5 hours	FastAPI + PostgreSQL + TypeScript	Product thinking + clean implementation

Objective

Badi eSIM is a product developed and operated by Digital Cloud Communications. Build a small support ticketing module for Badi eSIM. Customers should be able to open and follow a support case, while support agents should be able to manage, reply to, assign and resolve tickets. The goal is not visual perfection; we want to see how you design the data model, APIs, workflows and user experience.

Scenario

Badi customers may contact support about installation, activation, connectivity, orders, top-ups, refunds or general questions. Today, imagine that Badi wants to introduce a simple in-product ticketing system so each case has a clear owner, status, conversation history and audit trail.

Core requirements

1. Ticket creation

- Allow a customer or support agent to create a new ticket.
- Capture: customer email, optional order ID, category, subject, description and priority.
- Generate a readable ticket number, for example BD-1001.
- New tickets should start in OPEN status.

```
Example ticket
Ticket: BD-1001
Customer: customer@example.com
Order ID: ORD-10293
Category: CONNECTIVITY
Subject: eSIM installed but no internet
Priority: HIGH
Status: OPEN
```

2. Ticket workflow

Support the following states and priorities:

States	Priorities
OPEN	LOW
IN_PROGRESS	MEDIUM

WAITING_FOR_CUSTOMER
WAITING_FOR_PROVIDER
RESOLVED
CLOSED

HIGH
URGENT

Suggested categories: INSTALLATION, ACTIVATION, CONNECTIVITY, ORDER, TOPUP, REFUND, OTHER.

3. Conversation and internal notes

- Each ticket must have a conversation history.
- Customers and support agents can add replies.
- Customer-visible replies must update in real time in both directions without a page refresh: customers see new agent replies, and agents see new customer replies. Typing indicators, presence and read receipts are out of scope.
- Support agents can add INTERNAL notes that are never shown to the customer.
- Each message should record sender, type and timestamp.

```
10:30 Customer
My eSIM is installed but internet is not working.

10:35 Support
Are you currently in Turkey?

10:40 Customer
Yes.

10:44 Internal note
Escalated to upstream provider for status verification.
```

4. Support agent actions

- View and filter tickets
- Open a ticket and review its full conversation
- Change status and priority
- Assign or reassign the ticket to an agent
- Send a customer-visible reply
- Add an internal note

5. Audit history

Important changes must be recorded so the team can see what happened and when. At minimum, record status changes and assignment changes.

```
10:45 Status changed: OPEN -> IN_PROGRESS
10:46 Assigned to: Anas
11:12 Priority changed: MEDIUM -> HIGH
```

Technical requirements

Backend

- Python + FastAPI

- PostgreSQL
- SQLAlchemy / SQLModel is preferred
- Database migrations
- Clear API validation and error handling

Minimum API surface:

```
POST    /tickets
GET     /tickets
GET     /tickets/{id}
PATCH  /tickets/{id}
POST    /tickets/{id}/messages
```

GET /tickets should support useful filters such as status, priority, category and assigned agent. Search is a plus.

Frontend

Build a simple support interface in React + TypeScript. React Native / Expo may be used for the customer-facing part if you want to demonstrate mobile experience.

- Ticket list with status, priority and category filters
- Ticket detail screen
- Conversation thread
- Reply composer with an Internal note option
- Controls for status, priority and assignee
- Clear loading, empty and error states

Optional Badi context

You may associate a ticket with an existing order and show a small read-only order summary. Use mock data or a mock endpoint; no real eSIM provider integration is required.

```
GET /mock/orders/ORD-10293

{
  "order_id": "ORD-10293",
  "destination": "Turkey",
  "package": "10 GB",
  "status": "completed",
  "esim_status": "installed"
}
```

Data model

Design the schema yourself. We expect a sensible separation of concerns. A typical solution may include tables such as tickets, ticket_messages and ticket_events/audit_logs, but you are free to choose a different design if you can explain it.

Testing

Include at least two automated tests. For example:

- Creating a ticket generates a ticket number and starts it in OPEN status.
- Changing ticket status creates the expected audit-history entry.

Submission

- Source code in a Git repository or ZIP archive.
- README with setup and run instructions.
- Database migrations and seed/demo data.
- Any assumptions or trade-offs you made.
- Complete AI coding transcript or session export, if AI tools were used.
- A short note on what you would improve with more time.

Evaluation

Area	Weight
PostgreSQL schema and data modelling	20%
FastAPI / API design	20%
Frontend implementation and usability	20%
Ticket workflow and audit trail	15%
Code structure and maintainability	10%
Automated testing	10%
README and explanation of decisions	5%

Bonus ideas

Optional bonus work may include a customer-facing React Native / Expo ticket screen, file attachments, reply notifications, an SLA/overdue indicator, Docker setup, pagination/search, or role-based access.

AI tools

You may use AI coding assistants. If you use one, you must include the complete AI coding transcript or session export with your submission. One-shot AI-generated submissions will not be considered. You must understand and be able to explain the important architectural decisions and the code you submit, and be prepared to make a small change live during the review.